

AI Smart Link Manager - Day 3 Detailed Development Report

This document records the complete Day 3 journey, including decisions, debugging, implementation steps, and final outcomes.

Project State Before Day 3

The project already had a React frontend, backend API communication, summary display area, and basic URL processing workflow. The goal of Day 3 was to transform it into a production-style full-stack AI application.

Step 1: Git and GitHub Workflow

Reviewed repository status, commits, pushes, and synchronization with GitHub. Learned why commits represent project checkpoints and why pushing frequently prevents loss of work.

Step 2: Gemini API Integration

Created a Gemini API key, configured environment variables, installed the SDK, initialized GoogleGenerativeAI, created models, generated summaries, and connected AI outputs to the frontend.

Concepts Learned

API Keys, SDKs, request-response architecture, prompt engineering, asynchronous operations, model selection, and AI service integration.

Step 3: AI Tag Generation

Extended Gemini usage beyond summaries. Learned how to request structured outputs, parse comma-separated responses, split strings into arrays, trim values, and display tags dynamically in React.

Step 4: Error Investigation

Encountered Gemini 503 and 429 errors. Learned the difference between server overload and quota exhaustion. Added try-catch blocks and analyzed API responses.

Step 5: MySQL Integration

Created database, table schema, installed mysql2, configured connections, tested connectivity, executed queries, inserted rows, and validated records using SQL.

Step 6: Database Schema Design

Created links table with id, original_url, short_url, summary, tags, and created_at. Learned primary keys, auto-increment columns, text storage, and timestamps.

Step 7: Duplicate URL Detection

Implemented caching logic. Before calling Gemini, the application searches MySQL. Existing records are returned directly. New URLs trigger AI generation and database insertion.

Why This Matters

This reduces API usage, improves response speed, prevents duplicate rows, and demonstrates practical backend optimization.

Step 8: Environment Variables

Moved Gemini API key and database password into .env. Learned why secrets should never be committed to GitHub.

Step 9: README Creation

Documented project overview, features, workflow, database schema, learning outcomes, and future enhancements.

Final Architecture

React Frontend → Express Backend → MySQL Cache Check → Gemini API (if required) → Database Storage → Response to Frontend.

Day 3 Outcome

The project evolved from a simple application into a full-stack AI-powered system using React, Express.js, Gemini API, MySQL, Git, GitHub, SQL, and environment variables.